

Soldiers Surf For Free Operations Runbook

Version: 1.1 Last updated: 2026-06-08 Audience: maintainers, deployers, and operators Canonical source: this Markdown file. The DOCX version is a shareable artifact generated from this content.

Table of Contents

1. System Overview	2
2. Local Setup And Run	3
2.1 Prerequisites	3
2.2 Start Local Postgres	3
2.3 Backend Setup	3
2.4 Frontend Setup	4
2.5 Optional Local IoT/SQS Listener	4
2.6 Local Test And Quality Commands	5
3. Environment Variables	6
3.1 Backend Runtime Variables	6
3.2 Frontend Variables	7
3.3 AWS, IoT, And Local Tooling Variables	7
3.4 Database Seeding And Migration Variables	8
3.5 SQLAlchemy And Lambda Tuning Variables	8
3.6 Email Variables	9
3.7 GitHub Environment Secrets And Variables	9
4. AWS Architecture And IaC	11
4.1 IaC Source Of Truth	11
4.2 AWS Service Responsibilities	12
4.3 Runtime Flows	13
5. Deployment Guide	15
5.1 Branch And Environment Mapping	15
5.2 One-Time Cloud Preconditions	15
5.3 Backend Deploy Through CI	15
5.4 Manual Backend Deploy	16
5.5 Frontend Deployment In Amplify	17
5.6 Post-Deploy Verification	17
6. Database Maintenance	18
6.1 Local Reset	18
6.2 Export Current Seed	19
6.3 Seed Neon Or Another Remote Database	19
6.4 Production Migrations	19
7. CI/CD Maintenance	19
7.1 Workflow Responsibilities	19
7.2 Required GitHub Setup	20
7.3 Pre-Commit And Traceability	20
8. Security And Secret Handling	20
9. Hardware Setup Guide	21
10. Operational Checklists	26

10.1 Before First Dev Deploy	26
10.2 Before First Prod Deploy	26
10.3 Routine Deploy	26
10.4 Rollback	26
11. Troubleshooting	27
12. Maintenance Boundaries	27
13. Source References	28

1. System Overview

Soldiers Surf For Free is a full-stack locker and equipment rental system.

The repository contains:

- Backend: FastAPI, SQLAlchemy, Alembic, PostgreSQL, AWS Lambda container images.
- Frontend: React, Vite, Material UI, AWS Amplify Hosting.
- IoT path: AWS IoT Core receives locker MQTT events and triggers a Lambda handler.
- Local IoT/dev path: an optional SQS listener can process IoT events routed to a dev queue.
- IaC: AWS SAM/CloudFormation templates for the backend, API Gateway, Lambda, IoT rules, IAM permissions, and docs bucket.
- CI/CD: GitHub Actions for tests, frontend build, traceability, migrations, and backend deployment.

Primary entry points:

Area	Source
Backend app	backend/app/__init__.py, backend/wsgi.py
HTTP Lambda handler	backend/app/http_lambda.py
IoT Lambda handler	backend/app/lambda_function.py
Backend SAM template	template.yaml
Legacy IoT nested template	template-legacy-v1.yaml
Frontend app	frontend/src/main.tsx, frontend/src/App.tsx
Amplify build config	amplify.yml
GitHub CI	.github/workflows/tests.yml
GitHub backend deploy	.github/workflows/deploy-backend.yml
Local Postgres	docker-compose.yml
Infrastructure reports and bootstrap templates	infrastructure/

2. Local Setup And Run

2.1 Prerequisites

Install these before running the system locally:

- Python 3.12.
- Node.js 22, matching the GitHub workflow.
- Docker Desktop or a compatible Docker engine.
- Git.
- Optional for AWS/IoT work: AWS CLI, AWS SAM CLI, and valid AWS credentials.
- Optional for Git hooks: `pre-commit`.

2.2 Start Local Postgres

From the repository root:

```
docker compose up -d postgres
```

The local database defaults are:

Database	URL
App DB	postgresql+psycopg://ssf:ssf@localhost:5433/soldiers_surf_free
Test DB	postgresql+psycopg://ssf:ssf@localhost:5433/soldiers_surf_free_test

The test database is created by `docker/postgres/init/01-create-test-database.sql` when the Postgres volume is initialized.

2.3 Backend Setup

From the repository root:

```
cd backend
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

On Windows PowerShell, activate the venv with:

```
.\.venv\Scripts\Activate.ps1
```

Create a local env file from the example:

```
cp .env.example .env
```

Use local placeholder/dev values only. Do not commit `.env`.

Run migrations and load development seed data:

```
python scripts/setup_dev_db.py
```

Start the backend:

```
python -m uvicorn wsgi:app --reload
```

Health check:

```
curl http://127.0.0.1:8000/api/health
```

Expected response:

```
{"status":"ok"}
```

2.4 Frontend Setup

From the repository root:

```
cd frontend
npm ci
cp .env.example .env.local
```

For local development, set:

```
VITE_API_BASE_URL=http://127.0.0.1:8000/api/v1
VITE_COGNITO_DOMAIN=https://<cognito-domain>.auth.<region>.amazoncognito.com
VITE_COGNITO_CLIENT_ID=<cognito-app-client-id>
VITE_COGNITO_REDIRECT_URI=http://localhost:5173/auth/callback
VITE_COGNITO_LOGOUT_URI=http://localhost:5173
```

Start the frontend:

```
npm run dev
```

The Vite app normally runs at:

```
http://localhost:5173
```

2.5 Optional Local IoT/SQS Listener

Use this path only when testing the local worker that consumes AWS SQS messages from the dev IoT tee.

Required first:

- AWS credentials available through `AWS_PROFILE` or standard AWS credential env vars.
- `IOT_REGION` set.
- The dev SQS queue and IoT role exist or have been bootstrapped. The legacy bootstrap script is `python -m app.scripts.setup_aws_resources`; the CloudFormation adoption template is `infrastructure/bootstrap/dev-iot-sqs.yaml` and must not be deployed over existing resources without following `infrastructure/adoption-plan.md`.

From backend/:

```
python -m app.run_listener
```

The listener resolves the queue name from:

```
{PROJECT_PREFIX}_{IOT_SQS_QUEUE_NAME}
```

With defaults, that is:

```
soldiers_surf_free_locker_events
```

2.6 Local Test And Quality Commands

Backend tests:

```
cd backend  
python -m pytest -q
```

Regenerate test index without running tests:

```
cd backend  
python -m pytest --collect-only --gen-index -q --no-cov
```

Frontend lint and production build:

```
cd frontend  
npm run lint  
npm run build
```

The frontend build runs:

- required env-var validation via `frontend/scripts/check-env.mjs`;
- TypeScript build;
- Vite build;
- bundle budget check via `frontend/scripts/check-bundle-size.mjs`.

3. Environment Variables

General rule: treat any value containing a password, token, private key, session token, database credential, OAuth client secret, or signing/encryption material as secret. Do not commit `.env`, `.env.local`, real connection strings, AWS credentials, or generated ESP secrets.

3.1 Backend Runtime Variables

Variable	Secret	Required	Used by	Purpose
APP_ENV	No	Optional locally; set by SAM	Backend config	production enables fail-fast required settings. Local default is development. SAM currently sets this to production for deployed Lambdas.
DATABASE_URL	Yes for real DBs	Required in production	Backend, Alembic, CI deploy	Main PostgreSQL connection string. Local default points at Docker Postgres. Remote URLs usually contain credentials.
TEST_DATABASE_URL	Yes for real DBs	Required for CI/tests when not using default	Tests	PostgreSQL URL used by TestConfig and CI test runs.
CORS_ORIGINS	No	Required when APP_ENV=production	FastAPI CORS	Comma-separated allowed frontend origins. Local default includes <code>http://localhost:5173</code> and <code>http://localhost:3000</code> .
SECRET_KEY	Yes	Required in production	Auth flow	HMAC/signing secret for registration tokens and related backend security. Rotating invalidates pending registration tokens.
QR_SECRET_KEY	Yes	Required in production	QR/security flows	Signing secret for QR-related data. Defaults to SECRET_KEY only in non-production.
BLOWFISH_SECRET_KEY	Yes	Required where encrypted IDs are used	app.utils.crypto	Encrypts sensitive user identifiers. Rotating without a data migration can make existing encrypted IDs unreadable.
COGNITO_DOMAIN	No	Required for auth exchange	Backend auth router	Cognito Hosted UI domain used for <code>/oauth2/token</code> .
COGNITO_CLIENT_ID	No	Required for auth exchange	Backend auth router	Cognito app client ID.
COGNITO_CLIENT_SECRET	Yes	Required for auth exchange	Backend auth router	Cognito app client secret used for OAuth code exchange.
COGNITO_REDIRECT_URI	No	Required for auth exchange	Backend auth router	Redirect URI that must match the frontend callback configured in Cognito.
DOCS_BUCKET	No	Required for document upload flows	S3 utility	S3 bucket for temporary rental-session PDF/document objects. SAM sets <code>soldiers-surf-free-\${EnvironmentName}-docs</code> .
LOG_LEVEL	No	Optional	Logger utility	Backend log level. Defaults to INFO.

3.2 Frontend Variables

All Vite variables are exposed to browser code. None of these must contain secrets.

Variable	Secret	Required	Used by	Purpose
VITE_API_BASE_URL	No	Yes	Frontend API client	Backend API root ending /api/v1, for example https://<api-id>.execute-api.<region>.amazonaws.com/api/v1.
VITE_COGNITO_DOMAIN	No	Yes	Auth context	Cognito Hosted UI domain.
VITE_COGNITO_CLIENT_ID	No	Yes	Auth context	Cognito public app client ID.
VITE_COGNITO_REDIRECT_URI	No	Yes	Auth context	Frontend callback URL ending /auth/callback.
VITE_COGNITO_LOGOUT_URI	No	Yes	Auth context	Frontend root URL used after Cognito logout.
MODE / NODE_ENV	No	Optional	check-env.mjs	Selects which Vite env files are checked. Defaults to production behavior during build.

3.3 AWS, IoT, And Local Tooling Variables

Variable	Secret	Required	Used by	Purpose
AWS_PROFILE	No	Optional locally	AWS CLI, boto3 scripts	Named AWS profile for local commands. Prefer this over relying on an implicit default profile.
AWS_REGION	No	Required for CI deploy and local AWS commands	GitHub Actions, AWS CLI, setup scripts	AWS region for deployment and general AWS services. Use a placeholder like <region> in docs and configure the real value per environment.
AWS_ACCESS_KEY_ID	Yes	Optional locally	AWS SDK credential chain	AWS access key for local/lab credentials. Do not commit. Prefer named profiles for maintainers.
AWS_SECRET_ACCESS_KEY	Yes	Optional locally	AWS SDK credential chain	AWS secret key. Do not commit.
AWS_SESSION_TOKEN	Yes	Optional locally	AWS SDK credential chain	Temporary session token when using short-lived/lab credentials. Do not commit.
IOT_REGION	No	Required for IoT scripts/runtime	IoT publisher, SQS listener, setup scripts	Region for IoT Core and SQS operations. Defaults to us-east-1 in code if not set, so set it explicitly.
IOT_ENDPOINT	No	Optional deploy setting	IoT publisher, scripts	AWS IoT data-plane endpoint. If omitted, backend attempts iot:DescribeEndpoint.
TOPIC_PREFIX	No	Optional locally; dev deploy only	IoT topic builder	Prefix for outbound command topics. Code defaults to v1. SAM and CI now pass this only for dev (dev) and intentionally omit it for prod so live production remains on legacy v1 until a separately approved hardware/topic migration.

Variable	Secret	Required	Used by	Purpose
PROJECT_PREFIX	No	Optional	SQS listener/setup scripts	Prefix used to build local/dev SQS queue names. Default is soldiers_surf_free.
IOT_SQS_QUEUE_NAME	No	Optional	SQS listener/setup scripts	Base queue name. Default is locker_events.
IOT_SQS_POLL_WAIT_TIME	No	Optional	SQS listener	Long-poll wait time in seconds. Default is 20.
IOT_ROLE_ARN	No	Optional	AWS setup script	Existing IAM role ARN for IoT to publish to SQS. If omitted, setup script tries lab roles or creates {PROJECT_PREFIX}_iot_role.
MOCK_DEVICE_ID	No	Optional	Mock ESP script	Locker/device ID used by simulation scripts.

3.4 Database Seeding And Migration Variables

Variable	Secret	Required	Used by	Purpose
NEON_DATABASE_URL	Yes	Required for setup_neon_db.py unless passed as CLI arg	Neon seed script	Target Neon/Postgres URL for schema migration and seed loading.
SOURCE_DATABASE_URL	Yes for real DBs	Optional	Seed export script	Source DB URL for exporting current seed data. Defaults to DATABASE_URL, then local dev DB.

3.5 SQLAlchemy And Lambda Tuning Variables

Variable	Secret	Required	Used by	Purpose
SQLALCHEMY_ASYNC_POOL_SIZE	No	Optional	Database extension	Async DB pool size. SAM sets 1 for dev Lambda to match live dev and omits it for prod to preserve current prod behavior.
SQLALCHEMY_ASYNC_MAX_OVERFLOW	No	Optional	Database extension	Async DB pool overflow. SAM sets 2 for dev Lambda to match live dev and omits it for prod to preserve current prod behavior.
SQLALCHEMY_ASYNC_POOL_RECYCLE_SECONDS	No	Optional	Database extension	Connection recycle interval. SAM sets 300 for dev Lambda to match live dev and omits it for prod to preserve current prod behavior.
SQLALCHEMY_CREATE_SYNC_ENGINE	No	Optional	Database extension	Set truthy to create the sync engine in Lambda. Normally disabled in Lambda to reduce cold-start work.

3.6 Email Variables

Variable	Secret	Required	Used by	Purpose
EMAIL_USERNAME	Usually no, but sensitive	Required only when sending email	app.utils.email	Sender email address for SMTP.
EMAIL_PASSWORD	Yes	Required only when sending email	app.utils.email	SMTP/app password. Do not commit.

3.7 GitHub Environment Secrets And Variables

Create GitHub Environments named:

- `production`
- `development`

Environment secrets expected by `.github/workflows/deploy-backend.yml`:

Secret	Secret value	Purpose
AWS_ROLE_TO_ASSUME	Treat as protected deployment value	IAM role ARN trusted by GitHub OIDC. Stored as a secret because the workflow reads it from secrets.*.
DATABASE_URL	Yes	Environment-specific Postgres/Neon URL for migrations and Lambda runtime.
PROD_DATABASE_URL	Yes	Backward-compatible production DB secret. Prefer DATABASE_URL for new setup.
SECRET_KEY	Yes	Backend signing secret.
QR_SECRET_KEY	Yes	QR signing secret.
BLOWFISH_SECRET_KEY	Yes	Sensitive identifier encryption secret.
COGNITO_CLIENT_SECRET	Yes	Cognito app client secret.

Environment variables expected by `.github/workflows/deploy-backend.yml`:

Variable	Secret	Purpose
AWS_REGION	No	Region for AWS deploy.
BACKEND_STACK_NAME	No	Optional override. Defaults to <code>soldiers-surf-free-\$DEPLOY_ENV</code> .
COGNITO_ISSUER	No	Cognito issuer URL for API Gateway JWT authorizer.
COGNITO_AUDIENCE	No	JWT audience, usually the Cognito app client ID.
COGNITO_DOMAIN	No	Cognito Hosted UI domain.
COGNITO_CLIENT_ID	No	Cognito app client ID.

Variable	Secret	Purpose
COGNITO_REDIRECT_URI	No	Deployed frontend callback URL ending /auth/callback.
CORS_ALLOWED_ORIGINS	No	Comma-separated deployed frontend origins passed to SAM as CorsAllowedOrigins.
IOT_ENDPOINT	No	Optional IoT data endpoint override.

Computed by workflow; do not configure manually:

Variable	Source	Purpose
DEPLOY_ENV	github.ref_name == 'main' ? 'prod' : 'dev'	SAM EnvironmentName parameter.

4. AWS Architecture And IaC

4.1 IaC Source Of Truth

The deployable infrastructure for the backend is AWS SAM/CloudFormation. Use `template.yaml` as the application stack template and `template-legacy-v1.yaml` as the nested legacy IoT routing template. Do not use Terraform, CDK, or ad hoc console changes for resources that are already owned by these stacks.

The `infrastructure/` directory contains the operator-facing infrastructure reports and separate bootstrap/adoption templates. Those templates are not part of the normal backend deploy path. Use them only when performing the documented adoption procedure in `infrastructure/adoption-plan.md`.

Live environment map:

Item	Current value
Backend/core AWS account	817874646702
Backend/core AWS region	il-central-1
Amplify frontend region	eu-central-1
Dev SAM stack	soldiers-surf-free-dev
Prod SAM stack	soldiers-surf-free-prod
Prod nested legacy stack	soldiers-surf-free-prod-LegacyV1Stack-WMYXHE3CP8G1
Dev API root	https://5fczpatkof.execute-api.il-central-1.amazonaws.com
Prod API root	https://glwtzmyh6.execute-api.il-central-1.amazonaws.com
Amplify app	d3ft1grv9bjx57

SAM-managed application resources:

Area	Managed by	Operator notes
API Gateway HTTP API	template.yaml	Public API entry point, CORS, public routes, and Cognito JWT authorizer wiring.
API Lambda container	template.yaml	FastAPI/Mangum runtime. Prod builds as arm64; dev builds as x86_64.
IoT event Lambda container	template.yaml	Processes MQTT events forwarded by IoT topic rules. Prod builds as arm64; dev builds as x86_64.
Lambda IAM execution policies	template.yaml	Runtime permissions for logs, IoT publish/describe, and docs-bucket S3 operations.
Docs S3 bucket	template.yaml	Private per-environment bucket, retained by CloudFormation, with lifecycle expiration.
SAM-owned IoT topic rules	template.yaml	Routes environment-prefixed locker events to the IoT Lambda.

Area	Managed by	Operator notes
Legacy prod v1/locker/* routing	template-legacy-v1.yaml	Kept in prod until device topic migration is complete.
Prod locker 4 RFID republish rule	template.yaml	Existing production automation rule for locker 4.

Resources configured outside the SAM application stack:

Resource	Where it is configured	Operator rule
Cognito user pool and app clients	AWS Cognito console/API and GitHub environment vars/secrets	Reference the existing pool and clients through deploy parameters and frontend env vars. Do not create replacement pools from the app stack.
GitHub OIDC provider and deploy role	AWS IAM, GitHub Environments, .github/workflows/deploy-backend.yml	The workflow assumes github-actions-ssf-deploy. Use infrastructure/bootstrap/github-oidc.yaml only during an approved adoption.
IoT Things, X.509 certificates, and device policies	AWS IoT Core device provisioning process	Keep device private keys out of Git. Do not replace live certificates or shared device policy from the app stack.
Dev SQS queue and IoT-to-SQS role	Existing AWS resources or backend/app/scripts/setup_aws_resources.py	The queue soldiers_surf_free_locker_events is for dev/local listener workflows. Do not attach a new consumer until the backlog has been reviewed.
Amplify app, branches, branch env vars, rewrites, and headers	AWS Amplify console plus amplify.yml, customHttp.yml, and frontend/amplify-rewrites.json	Keep frontend branch settings aligned with the backend API/Cognito values for the matching environment.

4.2 AWS Service Responsibilities

Service	Responsibility
AWS Amplify Hosting	Builds and hosts the React/Vite frontend from frontend/ using amplify.yml.
Amazon API Gateway HTTP API	Public HTTPS entry point for backend API routes. Applies CORS and Cognito JWT authorization on protected /api/v1/{proxy+} routes.
AWS Lambda - API function	Runs FastAPI through Mangum using app.http_lambda.handler. Handles HTTP requests from API Gateway.
AWS Lambda - IoT function	Runs app.lambda_function.lambda_handler. Processes MQTT events forwarded by AWS IoT Core topic rules.
AWS IoT Core	Receives MQTT traffic from lockers/ESP32 devices, applies topic rules, invokes Lambda, and accepts backend command publishes.
AWS IoT Topic Rules	Route {env}/locker/{+}/+ events to Lambda, excluding cmd. Prod also keeps legacy v1/locker/{+}/+ routing until hardware migration is complete.
Amazon SQS	Dev-only tee for IoT events so the local listener can process messages. Not used by the prod stack path.
Amazon S3	Stores temporary PDF/document objects in the docs bucket. Objects are private and expire automatically.

Service	Responsibility
Amazon Cognito	Hosted UI and OAuth/OIDC identity provider for frontend login and backend code exchange. Must be created outside current SAM.
PostgreSQL/Neon	Primary relational database. CI runs Alembic migrations against DATABASE_URL before backend deploy.
AWS IAM	Lambda execution roles, IoT permissions, S3 permissions, GitHub OIDC deploy role, and optional IoT-to-SQS role.
Amazon ECR	Stores Lambda container images built by SAM. Repositories are resolved by <code>sam deploy --resolve-image-repos</code> .
AWS CloudFormation/SAM	Owns backend deployed infrastructure defined in <code>template.yaml</code> and <code>template-legacy-v1.yaml</code> .
Amazon CloudWatch Logs	Runtime logs for API Lambda, IoT Lambda, and deployment troubleshooting.

4.3 Runtime Flows

Local developer flow:

- Docker runs Postgres on local port 5433.
- Backend loads `backend/.env` outside Lambda.
- `python scripts/setup_dev_db.py` runs Alembic migrations and loads `backend/seeds/dev_data.sql`.
- Uvicorn serves FastAPI at `http://127.0.0.1:8000`.
- Vite serves React at `http://localhost:5173`.
- Frontend calls the backend through `VITE_API_BASE_URL`.

Frontend auth flow:

- User clicks login in the frontend.
- Frontend redirects to Cognito Hosted UI with `email openid profile` scopes.
- Cognito redirects back to `/auth/callback` with an authorization code.
- Frontend sends the code to backend `/api/v1/auth/exchange`.
- Backend exchanges the code with Cognito using `COGNITO_CLIENT_SECRET`.
- Backend returns either an authenticated user response or a registration-required token.

HTTP API flow in AWS:

- Browser calls Amplify-hosted frontend.
- Frontend calls `VITE_API_BASE_URL`.
- API Gateway receives the request.
- API Gateway allows `/api/health` and `/api/v1/auth/{proxy+}` without Cognito authorizer.
- API Gateway requires Cognito JWT auth for protected `/api/v1/{proxy+}` routes.
- API Gateway invokes the API Lambda.
- Mangum adapts API Gateway events to FastAPI.
- FastAPI uses async SQLAlchemy sessions against PostgreSQL.

IoT event flow in AWS:

- ESP32/locker publishes an MQTT event, reply, state, presence, or error message to an IoT topic.
- IoT topic rule adds `topic()` AS `topic` and `topic(3)` AS `locker_id`.
- IoT invokes the IoT Lambda.
- Lambda parses the event with `IotEventParser`.
- Backend derives trusted locker ID from the authenticated topic, not from untrusted payload claims.
- Backend updates DB state or creates follow-up commands.
- For actions such as RFID-approved open, backend publishes a command to IoT Core.
- Device receives the command on its subscribed `cmd` topic.

Backend command publish flow:

- API or IoT event handler creates an `IotCommand`.
- `AwsMqttProvider` builds topic `{TOPIC_PREFIX_OR_v1}/locker/{locker_id}/cmd`.
- Provider publishes through AWS IoT Data Plane with QoS 1.
- Device sends a reply event.
- Reply updates the matching `locker_commands` row by `request_id`.

Dev-only SQS listener flow:

- Dev IoT topic rule invokes Lambda and also sends a copy to SQS.
- Local `python -m app.run_listener` polls SQS.
- Listener parses each message, ignores command loopback messages, and calls the same `process_event` logic.
- Listener deletes messages even on failure to avoid a poison-message loop in dev.

CI/CD flow:

- Push or PR runs `.github/workflows/tests.yml`.
- Backend code quality runs `flake8` and `mypy`.
- Frontend build runs `npm ci` and `npm run build`.
- Backend tests run against a Postgres service.
- Traceability job regenerates test index/report and checks freshness.
- Push to `main`, `development`, or `dev` also runs `.github/workflows/deploy-backend.yml`.
- Deploy workflow runs backend gates, validates deployment config, assumes AWS OIDC role, runs Alembic migrations, builds SAM with the target `EnvironmentName`, and deploys CloudFormation. Dev deploys pass `TopicPrefix=dev`; prod deploys intentionally omit `TopicPrefix`.
- Amplify deploys frontend when the connected branch changes.

5. Deployment Guide

5.1 Branch And Environment Mapping

Git branch	GitHub environment	SAM EnvironmentName	Default stack name
main	production	prod	soldiers-surf-free-prod
development	development	dev	soldiers-surf-free-dev
dev	development	dev	soldiers-surf-free-dev

5.2 One-Time Cloud Preconditions

Before first deploy, confirm these prerequisites exist for the target environment:

- Create a production database and a separate development database.
- Create Cognito user pool and app clients for each environment.
- Configure Cognito callback/logout URLs for the deployed Amplify domains.
- Create GitHub Environments `production` and `development`.
- Add required GitHub secrets and vars listed in Section 3.7.
- Create or verify the GitHub OIDC IAM role trusted by the repo environments. In the current live account, the workflow uses `github-actions-ssf-deploy`.
- Configure AWS IoT Things, certificates, and device policies.
- For dev SQS tee, use the live queue if it already exists. If bootstrapping a new non-production account from scratch, run the legacy script from `backend/`:

```
python -m app.scripts.setup_aws_resources
```

- Confirm S3 bucket names in `template.yaml` are available, or change the template before first deploy.
- Remove pre-existing manually created IoT topic rules that have the same names as SAM-managed rules, otherwise CloudFormation create can fail.
- Do not deploy, import, delete, or replace existing production resources until the CloudFormation change set and adoption plan have been reviewed.

5.3 Backend Deploy Through CI

Normal backend deploy is CI-driven:

```
push to development/dev -> deploy dev stack
push to main             -> deploy prod stack
```

The deploy workflow performs:

- checkout;
- Python dependency install;
- `flake8`, `mypy`, and `pytest`;
- required deploy config validation;
- AWS credential configuration through OIDC;
- Alembic upgrade against `DATABASE_URL`;

- `sam build --use-container --template-file template.yaml --parameter-overrides EnvironmentName="$DEPLOY_ENV";`
- `sam deploy` with:
 - `CAPABILITY_IAM;`
 - `CAPABILITY_NAMED_IAM;`
 - `CAPABILITY_AUTO_EXPAND;`
 - `--resolve-s3;`
 - `--resolve-image-repos;`
 - environment-specific parameter overrides, including `TopicPrefix=dev` only for dev.

5.4 Manual Backend Deploy

Use manual deploys for first deployment, controlled changes, or recovery.

From the repository root:

```
export AWS_PROFILE=<profile>
export AWS_REGION=<region>
aws sts get-caller-identity
```

Confirm the identity is the intended deploy identity before proceeding.

Build the same environment you intend to deploy. The build parameter matters because the Lambda image architecture differs by environment:

```
sam build --use-container --template-file template.yaml \
  --parameter-overrides EnvironmentName=dev
```

Deploy dev:

```
sam deploy \
  --template-file .aws-sam/build/template.yaml \
  --stack-name soldiers-surf-free-dev \
  --region <region> \
  --resolve-s3 \
  --resolve-image-repos \
  --confirm-changeset \
  --capabilities CAPABILITY_IAM CAPABILITY_NAMED_IAM CAPABILITY_AUTO_EXPAND \
  --parameter-overrides \
    EnvironmentName=dev \
    DatabaseUrl='<dev-database-url>' \
    SecretKey='<secret-key>' \
    QrSecretKey='<qr-secret-key>' \
    BlowfishSecretKey='<blowfish-secret-key>' \
    CognitoClientSecret='<cognito-client-secret>' \
    CognitoIssuer='https://cognito-idp.<region>.amazonaws.com/<user-pool-id>' \
    CognitoAudience='<cognito-app-client-id>' \
    CognitoDomain='https://<cognito-domain>.auth.<region>.amazoncognito.com' \
    CognitoClientId='<cognito-app-client-id>' \
    CognitoRedirectUri='https://<frontend-domain>/auth/callback' \
    CorsAllowedOrigins='https://<frontend-domain>' \
    IotEndpoint='https://<iot-data-endpoint>' \
    TopicPrefix=dev
```


Deploy prod by changing:

- stack name to `soldiers-surf-free-prod`;
- `EnvironmentName=prod`;
- all env-specific parameters to production values;
- rebuild with `--parameter-overrides EnvironmentName=prod`;
- omit `TopicPrefix` unless a separate production topic migration is approved.

5.5 Frontend Deployment In Amplify

Connect AWS Amplify Hosting to the GitHub repository.

Configure branches:

Branch	Environment
main	Production frontend
development or dev	Development frontend

Amplify uses `amplify.yml`:

- app root: `frontend`;
- pre-build: `npm ci`;
- build: `npm run build`;
- artifacts: `frontend/dist`.

Set Amplify branch env vars:

Variable	Value shape
VITE_API_BASE_URL	<code>https://<api-id>.execute-api.<region>.amazonaws.com/api/v1</code>
VITE_COGNITO_DOMAIN	<code>https://<cognito-domain>.auth.<region>.amazoncognito.com</code>
VITE_COGNITO_CLIENT_ID	Cognito app client ID for that environment
VITE_COGNITO_REDIRECT_URI	<code>https://<frontend-domain>/auth/callback</code>
VITE_COGNITO_LOGOUT_URI	<code>https://<frontend-domain></code>

Add the SPA rewrite rule from `frontend/amplify-rewrites.json` in Amplify Rewrites and redirects. Add custom headers matching `customHttp.yml` if Amplify is not already using that file in your setup.

5.6 Post-Deploy Verification

Check CloudFormation:

```
aws cloudformation describe-stacks \  
  --stack-name <stack-name> \  
  --query 'Stacks[0].StackStatus'
```

Check backend health:

```
curl https://<api-id>.execute-api.<region>.amazonaws.com/api/health
```

Expected:

```
{"status":"ok"}
```

Check stack outputs:

```
aws cloudformation describe-stacks \  
  --stack-name <stack-name> \  
  --query 'Stacks[0].Outputs'
```

Check IoT rule:

```
aws iot get-topic-rule --rule-name ssf_<env>_locker_events
```

Check Lambda logs:

```
aws logs describe-log-groups --log-group-name-prefix /aws/lambda/
```

Check docs bucket lifecycle:

```
aws s3api get-bucket-lifecycle-configuration \  
  --bucket soldiers-surf-free-<env>-docs
```

6. Database Maintenance

6.1 Local Reset

From the repo root:

```
docker compose up -d postgres
cd backend
python scripts/setup_dev_db.py
```

This upgrades the schema and reloads `backend/seeds/dev_data.sql`.

6.2 Export Current Seed

From `backend/`:

```
python scripts/export_current_seed.py --output seeds/neon_seed.sql
```

The source DB defaults to:

- `SOURCE_DATABASE_URL`;
- `DATABASE_URL`;
- local dev database.

6.3 Seed Neon Or Another Remote Database

From `backend/`:

```
NEON_DATABASE_URL='<remote-postgres-url>' python scripts/setup_neon_db.py --yes
```

Important:

- The script truncates target tables before loading the seed.
- It refuses localhost.
- It expects a Neon host unless `--allow-non-neon` is passed.
- Use separate databases or branches for development and production.

6.4 Production Migrations

CI runs Alembic before SAM deploy. For a manual migration, use the same logic as the workflow:

```
cd backend
DATABASE_URL='<remote-postgres-url>' python - <<'PY'
import os
from alembic import command
from alembic.config import Config

alembic_cfg = Config("alembic.ini")
alembic_cfg.set_main_option("sqlalchemy.url", os.environ["DATABASE_URL"])
command.upgrade(alembic_cfg, "head")
PY
```

7. CI/CD Maintenance

7.1 Workflow Responsibilities

Workflow	Trigger	Responsibility
.github/workflows/tests.yml	Push and PR to any branch	Backend quality, frontend build, backend tests, traceability report.
.github/workflows/deploy-backend.yml	Push to main, development, dev; manual dispatch	Backend gates, migrations, SAM build, SAM deploy.

7.2 Required GitHub Setup

Repository settings:

- Create environments `production` and `development`.
- Restrict `production` to deploy from `main`.
- Restrict `development` to deploy from `development` and/or `dev`.
- Add environment secrets and vars from Section 3.7.
- Verify the OIDC deploy role trust policy allows the relevant environment subjects:

```
{
  "StringLike": {
    "token.actions.githubusercontent.com:sub": [
      "repo:<owner>/<repo>:environment:production",
      "repo:<owner>/<repo>:environment:development"
    ]
  }
}
```

7.3 Pre-Commit And Traceability

Install pre-commit from the repo root:

```
pre-commit install
```

The local hook regenerates and validates:

```
backend/tests/docs/test_index.json
```

If test metadata changes, run:

```
cd backend
python -m pytest --collect-only --gen-index -q --no-cov
```

Commit the updated test index when it changes intentionally.

8. Security And Secret Handling

Rules:

- Never commit `.env`, `.env.local`, AWS credentials, database URLs, Cognito client secrets, SMTP passwords, ESP private keys, or generated device secrets.
- Keep production and development secrets separate.
- Rotate `SECRET_KEY`, `QR_SECRET_KEY`, and `BLOWFISH_SECRET_KEY` only with a clear migration/impact plan.
- Treat remote `DATABASE_URL` values as secrets because they contain credentials.
- Treat `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN` as credential material.
- Frontend `VITE_*` variables are public by design. Do not place secrets in them.
- For ESP32 production hardware, use AWS IoT X.509 certificates and private keys, not IAM user access keys.
- If a secret is accidentally committed, rotate it immediately and remove it from Git history if required.

9. Hardware Setup Guide

We have created a local admin/manager app to properly set up the Locker Hardware & connect to the system and AWS Infra. The app itself is interactive and can talk straight to AWS Infra!

<https://drive.google.com/drive/folders/1UqghDhPyhMnTYhpKwE9WCjcFd5WILA9F?usp=sharing>

- Attached guide at next page.

Soldiers Surf Free — Locker Admin Guide

OPERATOR GUIDE · DEVICE FLASHING, PROVISIONING & FIRMWARE UPDATES

First-time Setup (once)

1. Install **Python 3** from `python.org`.
2. Configure your **AWS credentials** on this computer (run `aws configure`, or set up the AWS profile you'll name in Settings). The app uses your machine's AWS access and never stores AWS keys.
3. Double-click `start.bat` (Windows) or run `./start.sh` (Mac/Linux) — installs dependencies (Flask + boto3) and opens the app at `http://localhost:8080`.
4. The app ships with a ready-to-use configuration, so this is usually just a check. Open the **Settings** tab and **verify** the AWS profile / region / project prefix / S3 bucket — they're normally pre-filled and need no changes. Only edit and **Save Settings** if something differs (e.g. a different AWS profile name). Either way, make sure your machine has AWS credentials for that profile (step 2).

Flashing a blank device (first time)

1. The supplied **core image** (a single `.bin`) is already in the `Scripts/aws/core/` folder — the Flash Device tab confirms it's present. (Only if it's missing, drop the supplied `.bin` into that folder.)
2. Connect the ESP32 to this computer over USB (install its CP210x/CH340 driver if Windows doesn't detect it).
3. **Flash Device** tab → click **Refresh ports**, pick the device's serial port, then click **Flash Device**. Wait for SUCCESS.
4. The device reboots into provisioning mode (blue LED). Continue with "Provisioning a new device" below.

You only flash the core image once per device. Day-to-day firmware updates are done over-the-air from the Firmware Update tab — no USB needed.

Provisioning a new device (two steps)

Step 1 is done at your desk (needs internet/AWS). Step 2 is done next to the physical locker.

1. **Provision Device** tab → choose environment (**dev** / **prod**), enter the **Locker ID**, choose the **Number of Cells** (1–2), and enter the beach WiFi **name**. (The WiFi password is entered in Step 2, not here.)
2. Click **Create AWS Identity** — registers the device in AWS and generates its certificates (saved under `Scripts/aws/certs/<env>/<id>/`). Wait for SUCCESS.
3. At the locker: power it on and **check the LED first**. A freshly flashed device enters **provisioning mode automatically** — the LED should already be blinking blue, so you can go straight to the next step. Only if it is *not* in provisioning (e.g. you're re-provisioning a device that's already configured) hold the **BOOT** button ~5 s until the LED blinks blue. If it stays unresponsive, power-cycle it (see Common Problems) rather than pressing RESET.
4. On your laptop, switch WiFi to the device's own network `SSF-Setup-XXXX` (WiFi password: `password`) — not your home WiFi. You must be on the device's network for the next step to reach it.
5. Enter the beach WiFi **password**, then click **Send to Device** — pushes the certificates, settings, cell count and WiFi to the device. Wait for SUCCESS.
6. Reconnect your laptop to normal WiFi. The device reboots and connects to AWS on its own.

The **Locker ID** and **environment** are fixed for the device's lifetime (they're its AWS identity — changing them means a new device). The **WiFi** and **number of cells** can be changed later — see below.

Updating an existing device (WiFi / cell count)

1. On the **Provision Device** tab, enter the **same environment and Locker ID** as the existing device, then set the new WiFi name and/or Number of Cells.
2. Click **Update Existing Device** — this edits the saved config (no new Thing or certificates). If that env + id doesn't exist yet, you'll get an error telling you to create it first.
3. Put the device in provisioning mode first — an already-configured device won't enter it on its own, so hold the **BOOT** button ~5 s until the LED blinks blue. Then do **Step 2 — Send to Device** (enter the WiFi password) to push the updated settings to the hardware. The device reboots with the new params.

Updating firmware (upload, then push)

1. Get the built firmware file from the dev team and **name it with its version**, e.g. `v0.0.8-eboxlock.bin`. Put it in the `Scripts/aws/firmware/` folder.
2. **Firmware Update** tab → under "Upload Firmware Image" click **Refresh list**, pick the file, and click **Upload to S3**. Wait for SUCCESS. (The version is taken from the filename.)
3. Under "Create Firmware Update Job", choose the environment and pick the version from the dropdown (click **Refresh images** if needed).
4. Choose **Single locker** (enter its ID) or **All lockers in environment**.
5. Click **Create OTA Job**. Each targeted device downloads and installs the update on its next boot/check-in.

A group update applies to the lockers in that environment *right now*; a device added later won't auto-update — create a single-locker job for it. The download link is valid for 24 hours.

Environment separation

- **dev** — test devices, development.
- **prod** — real deployed lockers at the beach.
- dev and prod are isolated: a dev device only talks on `dev/...` channels, prod on `prod/...`. A dev locker and a prod locker can even share the same number — their identities and certificates are separate.

Common Problems

Provisioning fails with "group not found"

The AWS device groups for that environment haven't been created. This is a one-time backend setup done by the dev team (CI/CD). Contact them before provisioning new devices.

No versions in the firmware dropdown

Nothing is in S3 yet. Drop a `.bin` into `Scripts/aws/firmware/`, use **Upload to S3** first, then click **Refresh images**.

"Could not list..." or other AWS errors

Your AWS credentials aren't set up or lack permission. Run `aws configure` for the profile named in Settings (needs IoT + read/write on the firmware S3 bucket).

Device doesn't connect after "Send to Device"

Make sure your laptop was on the device's `SSF-Setup-XXXX` WiFi when you clicked, and that the beach WiFi name/password are exactly right (case-sensitive).

Device is unresponsive or won't enter provisioning

Power-cycle it: **unplug the power and plug it back in**, rather than pressing the on-board RESET/EN button — a reset doesn't fully clear the ESP32's state, but a power cycle does. Then check the LED again (a freshly flashed device should come up blinking blue on its own).

10. Operational Checklists

10.1 Before First Dev Deploy

- Development database exists.
- Cognito dev app client exists.
- GitHub `development` environment has required secrets and vars.
- OIDC role trust allows `environment:development`.
- Dev SQS queue and IoT role are bootstrapped if the dev SQS tee is required.
- If adopting the existing dev SQS queue, inspect the backlog and keep consumers detached until the adoption plan is approved.
- SAM-owned IoT rule names are not already occupied by manual rules.
- Amplify development branch has correct `VITE_*` values.

10.2 Before First Prod Deploy

- Production database exists and is separate from development.
- Cognito production app client exists.
- GitHub `production` environment has required secrets and vars.
- OIDC role trust allows `environment:production`.
- Production frontend callback/logout URLs are configured in Cognito.
- IoT devices and policies are ready.
- Legacy `v1` routing is intentionally kept until all hardware is migrated.
- Production `TOPIC_PREFIX` is intentionally omitted so commands continue to use legacy `v1`. Do not set it without an approved hardware/topic migration.

10.3 Routine Deploy

- Merge to `development` or `dev` for dev deploy.
- Verify CI tests and deploy workflow pass.
- Smoke test `/api/health`.
- Smoke test an authenticated flow.
- Check CloudWatch logs for API and IoT Lambdas.
- Merge to `main` for prod deploy after dev verification.

10.4 Rollback

Automatic rollback:

- Failed `sam deploy` rolls CloudFormation back to the previous stable stack state.
- Wait for stack status to settle to `UPDATE_ROLLBACK_COMPLETE` if needed.

Manual rollback after a successful but bad deploy:

```
git revert <bad-commit-sha>
git push origin <deploy-branch>
```

CI redeploys the reverted version.

Emergency IoT fallback:

- If legacy rules were disabled but not deleted during a migration soak, re-enable them in the IoT console while diagnosing.

- Do not delete legacy routing until hardware migration is complete and stable.

11. Troubleshooting

Symptom	Likely cause	Fix
VITE_API_BASE_URL is not set	Missing frontend env vars	Set required VITE_* vars in .env.local or Amplify branch env.
Frontend build fails in Amplify	Missing Vite env var or bundle size check failure	Review Amplify env vars and frontend/scripts/check-bundle-size.mjs output.
Backend startup fails with missing secret	APP_ENV=production without required vars	Set production backend secrets and CORS_ORIGINS/SAM parameters.
Local backend cannot connect to DB	Postgres not running or wrong port	Run docker compose up -d postgres; confirm port 5433.
Tests cannot connect to Postgres	Test DB missing or URL wrong	Recreate Docker volume or verify TEST_DATABASE_URL.
CI deploy validation fails	Missing GitHub secret/var	Add the missing value to the correct GitHub Environment.
Requires capabilities during SAM deploy	Missing CloudFormation capabilities	Include CAPABILITY_IAM CAPABILITY_NAMED_IAM CAPABILITY_AUTO_EXPAND.
IoT rule create fails with resource already exists	Manual rule has same name as SAM rule	Delete or rename the unmanaged rule before first SAM deploy.
Lambda cannot publish to IoT	Missing IoT permissions or endpoint issue	Confirm IAM policy grants iot:Publish; set or verify IOT_ENDPOINT.
Old hardware stops receiving commands	Command topic prefix changed too early	Keep default v1 until devices subscribe to the new prefix.
Dev command topics still use v1	Dev build/deploy did not receive EnvironmentName=dev and TopicPrefix=dev	Rebuild with sam build --parameter-overrides EnvironmentName=dev and deploy with TopicPrefix=dev; review the change set before execution.
Prod command topics still use v1	Expected current behavior	Prod intentionally omits TOPIC_PREFIX. Only change this through an approved hardware/topic migration.
Dev SQS listener cannot find queue	Queue not bootstrapped or prefix/name mismatch	Run python -m app.scripts.setup_aws_resources; check PROJECT_PREFIX and IOT_SQS_QUEUE_NAME.
setup_neon_db.py refuses target	Host is localhost or non-Neon	Use the correct remote URL, or pass --allow-non-neon intentionally.
Existing encrypted IDs cannot be read	BLOWFISH_SECRET_KEY changed	Restore old key or run a planned decrypt/re-encrypt migration.

12. Maintenance Boundaries

Use these boundaries when operating or changing the system:

Area	Maintenance rule
Backend application infrastructure	Change template.yaml and template-legacy-v1.yaml, run sam validate, run sam build for the target environment, then review the CloudFormation change set before execution.
Shared/bootstrap infrastructure	Use templates under infrastructure/bootstrap/ only with the import/adoption procedure in infrastructure/adoption-plan.md.
Cognito	Maintain the existing user pool, app clients, Hosted UI domain, callback URLs, logout URLs, and client secrets outside the SAM app stack. Keep backend GitHub vars/secrets and frontend Amplify env vars aligned with Cognito.
GitHub OIDC deploy identity	Keep GitHub Environments, AWS_ROLE_TO_ASSUME, and the IAM role trust policy aligned. Do not switch the workflow to long-term AWS access keys.
IoT devices	Provision Things, certificates, private keys, and device policies through the IoT device process. Never commit device private keys or generated device secrets.
IoT topic migration	Prod command publishes use legacy v1 unless TOPIC_PREFIX is explicitly added to prod deploy parameters. Keep legacy routing active until devices are migrated and verified.
Dev SQS listener	The local listener consumes and deletes messages from the dev queue. Inspect queue depth before attaching a new consumer or replaying messages.
Amplify frontend	Keep Amplify branch env vars in sync with the matching backend API and Cognito app client. Keep SPA rewrites and headers aligned with frontend/amplify-rewrites.json and customHttp.yml.
Secrets	Store runtime secrets in GitHub environment secrets or operator-managed secret stores. Do not commit secrets, real DB URLs, Cognito client secrets, AWS credentials, SMTP passwords, or device keys.

13. Source References

This runbook is based on the current repository files:

- `README.md`
- `RUN_INSTRUCTIONS.md`
- `DEPLOYMENT.md`
- `backend/README.md`
- `backend/docs/DEPLOYMENT.md`
- `backend/.env.example`
- `frontend/.env.example`
- `docker-compose.yml`
- `template.yaml`
- `template-legacy-v1.yaml`
- `infrastructure/current-architecture.md`
- `infrastructure/resource-inventory.md`
- `infrastructure/adoption-plan.md`
- `infrastructure/ci-cd-integration.md`
- `infrastructure/unmanaged-resources.md`
- `infrastructure/bootstrap/*.yaml`
- `amplify.yml`
- `customHttp.yml`
- `frontend/amplify-rewrites.json`
- `.github/workflows/tests.yml`
- `.github/workflows/deploy-backend.yml`
- `backend/app/config.py`
- `backend/app/extensions.py`
- `backend/app/http_lambda.py`
- `backend/app/lambda_function.py`
- `backend/app/routers/auth.py`
- `backend/app/routers/esp.py`
- `backend/app/service/providers/aws_mqtt.py`
- `backend/app/service/providers/sqs_listener.py`
- `backend/app/utils/aws_handler.py`
- `backend/app/utils/crypto.py`
- `backend/app/utils/email.py`
- `backend/app/utils/iot_topics.py`
- `frontend/src/api/config.ts`
- `frontend/src/context/AuthContext.tsx`